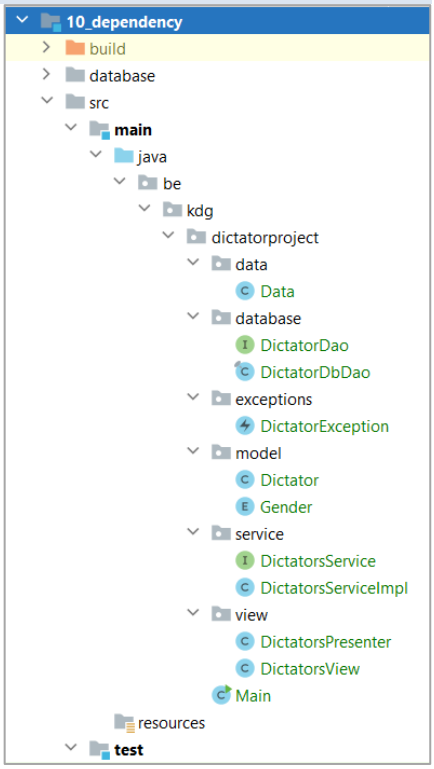
GROEIPROJECT
MODULE 10: INTEGRATIE & DEPENDENCY INJECTION



1. Voorbereiding



1.1. Open het groeiproject en maak daarin een **tiende** module: `m10_dependency`

1.2. We bouwen de hiernaast afgebeelde packagestructuur uit → Kopieer alvast vanuit module `m6_persistentie` de volgende klassen en interfaces: `XxxDao` , `XxxDbDao` , de `Data`-klasse en de basisklasse. De multiklasse heb je deze keer niet nodig.

1.3. Voor de `build.gradle` baseer je je best op de slides. Je hebt oa **javafx**, **hsqldb** en **junit** nodig.

1.4. In wat volgt werk je een **managementsysteem** uit naar analogie van het "Student Management System"-voorbeeld dat werd besproken en dat je ook terugvindt op gitLab.

2. Integratie & Dependency Injection

2.1. De interface `DictatorDao` vul je aan met de methode `getAllXxx` (in ons voorbeeld: `getAllDictators`) die een List retourneert. Werk de methode ook uit in de klasse `XxxDbDao` (maak zoveel mogelijk gebruik van bestaande methoden).

2.2. Maak een aparte klasse `XxxException` (in ons voorbeeld `DictatorException`). Je maakt een **unchecked** Exception; dus overerven van `RuntimeException`. Voorzie 5 constructors, naar analogie met de superklasse.

2.3. Pas de klasse `XxxDbDao` als volgt aan:

a) Verander de methode `createTable`: indien de database tabel nog niet bestaat, dan wordt hij aangemaakt én gevuld met de elementen uit de klasse `Data`.

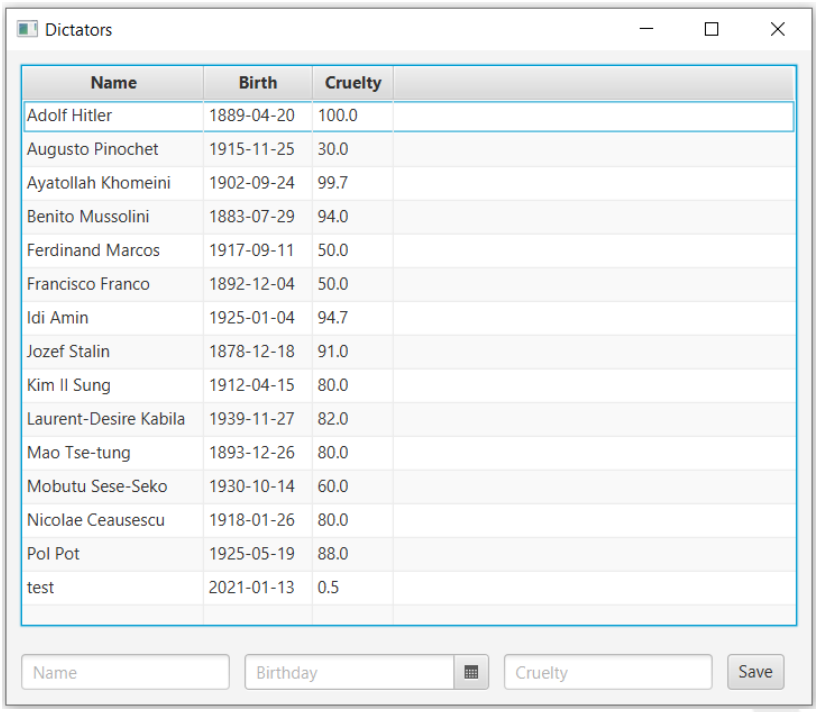
b) Pas het **singleton** design pattern toe op de klasse `XxxDbDao`

c) Voorzie een `Logger`-object dat je gebruikt in elke catch om de fout te loggen (level: **warning**) en vervolgens te wrappen in je eigen `XxxException` en verder te throwen.

d) Voorzie ook logging van het level **info** op belangrijke plaatsen waar alles OK is (bijvoorbeeld: "Tabel aangemaakt en gevuld met data").

2.4. In de package `service` maak je een interface `XxxService` (in ons voorbeeld `DictatorsService`) en ook een implementerende klasse `XxxServiceImpl` (in ons voorbeeld: `DictatorsServiceImpl`). Deze laatste maakt intern gebruik van de Dao-klasse.
a) Voorzie een methode `getAllXxx` (in ons voorbeeld `getAllDictators`)
b) Voorzie een methode `addXxx` (in ons voorbeeld `addDictator`)
c) Zorg voor dependency injection: benader dus de Dao-klasse niet rechtstreeks, maar wel via de interface!

2.5. We passen het **model-view-presenter** (MVP) principe toe. Maak eerst de **viewklasse** (in ons voorbeeld: `DictatorsView`):
a) Laat overerven van `BorderPane`
b) Voorzie een `TableView`, een aantal `TextFields` en een `DatePicker`.
c) Probeer onderstaande layout te benaderen. Het volstaat dat je 3 eigenschappen toont in de TabelView: een String, een LocalDate en een double of een int (in ons voorbeeld hieronder: name, birth en cruelty). Het moet voor de gebruiker ook mogelijk zijn om een nieuw element toe te voegen via de `TextFields` en de `DatePicker` onderaan. Voorzie eventueel in je basisklasse een extra constructor op basis van deze 3 eigenschappen.
d) Voorzie in de viewklasse ook package-private getters voor deze componenten, zodat de presenter (die in dezelfde package zit) de belangrijke componenten kan benaderen.



2.6. Maak de **presenterklasse** (in ons voorbeeld: `DictatorsPresenter`):
a) Via de constructor komen de viewklasse en de serviceklasse binnen.
b) Doe de eventhandling van de Save-button
c) Zorg dat de `TableView` bij elke wijziging opnieuw gerefresht wordt.
d) Catch de gethrowde exceptions en toon de bijhorende foutmelding in een `Alert`-venster.

2.7. Maak tenslotte de klasse `Main`:
a) In de methode `start` instantieer je de dao, de service, de view en de presenter.
b) Denk aan het principe van **dependency injection**: gebruik zoveel mogelijk de interfaces!
c) Zet je view in een Scene en toon die op de primaryStage.

Alles klaar? Run via Gradle en controleer! Gebruik eventueel de database tool om de inhoud van de databank te bekijken.



Proficiat!!! Je groeiproject is nu volledig klaar!!!

Doe een laatste commit naar GitLab en geef jezelf een welverdiend schouderklopje…

